Guin-16

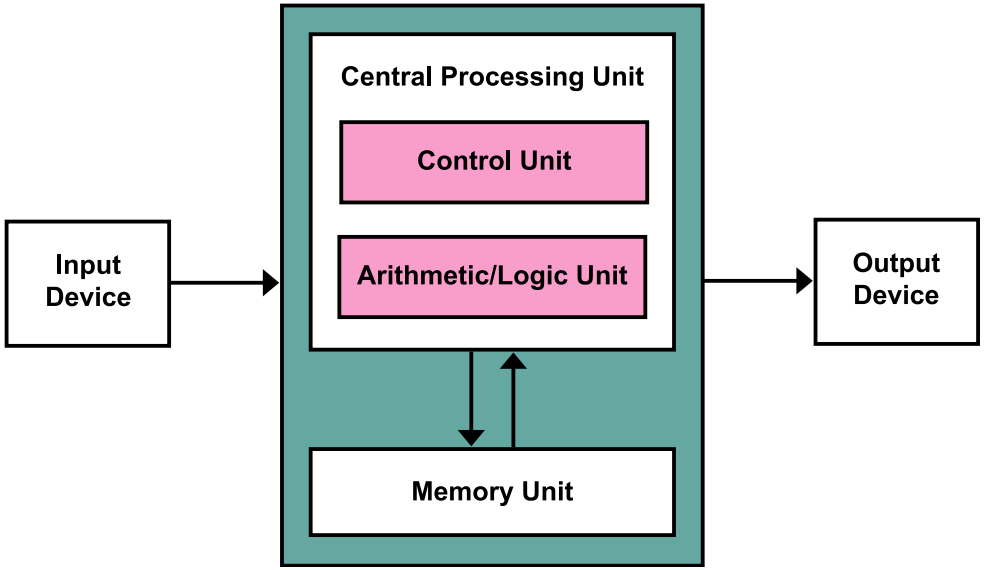
An Original 16-Bit CPU

Jacob Shogren

https://github.com/shogrenjacob/16-bit-cpu

Overview

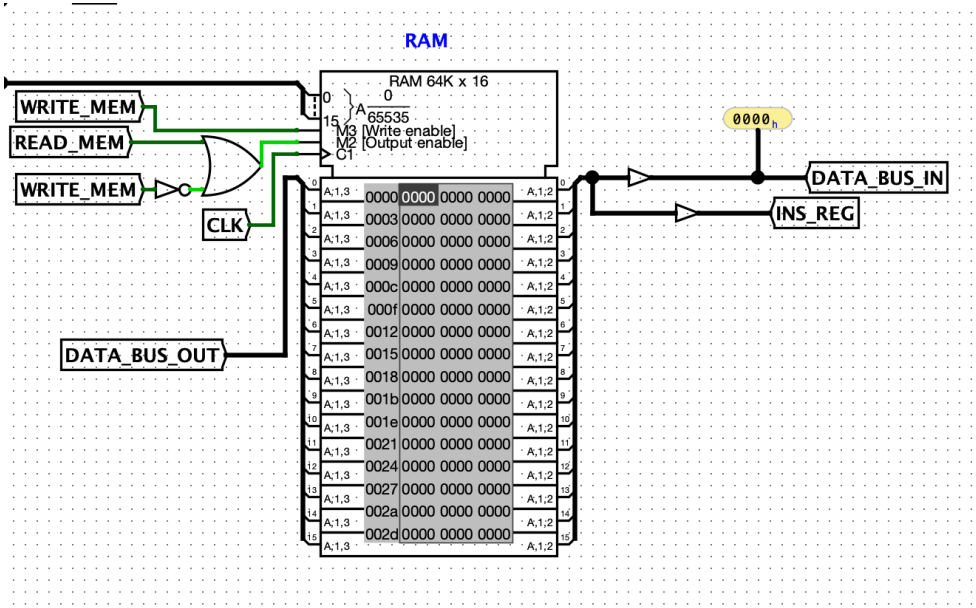
- Von Neumann Architecture
- 65,535 memory addresses (255 per page)
- 7 Registers, 5 Flags
- Two's Compliment
- Microcode Instruction Set Implementation



By Kapooht - Own work, CC BY-SA 3.0,
https://commons.wikimedia.org/w/index.php?curid=25789639

Memory

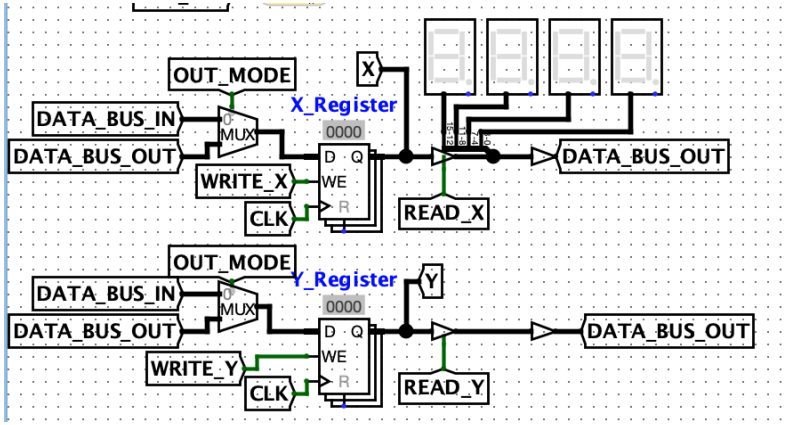
- Stack Memory (0x00 → 0xff)
- Free Memory (0x100 → 0xffff)

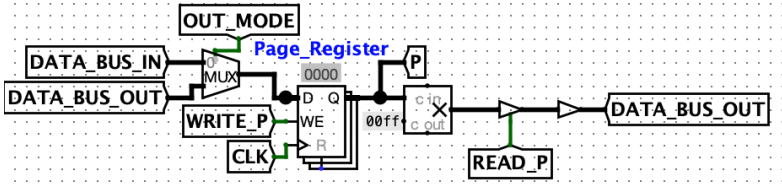


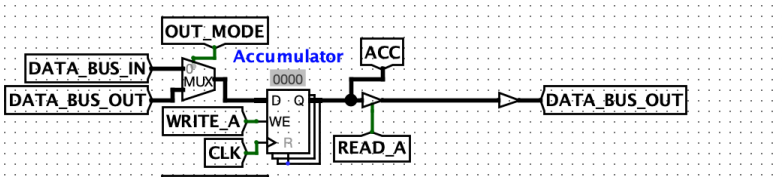
0×00 - 0xFF (Stack Memory)	0×100 - 0xFFFF (Free Memory)

Registers

- General-Purpose Registers
- Accumulator
- Instruction Register
- Page Register
- Flag Register
- Memory Address Register (MAR)

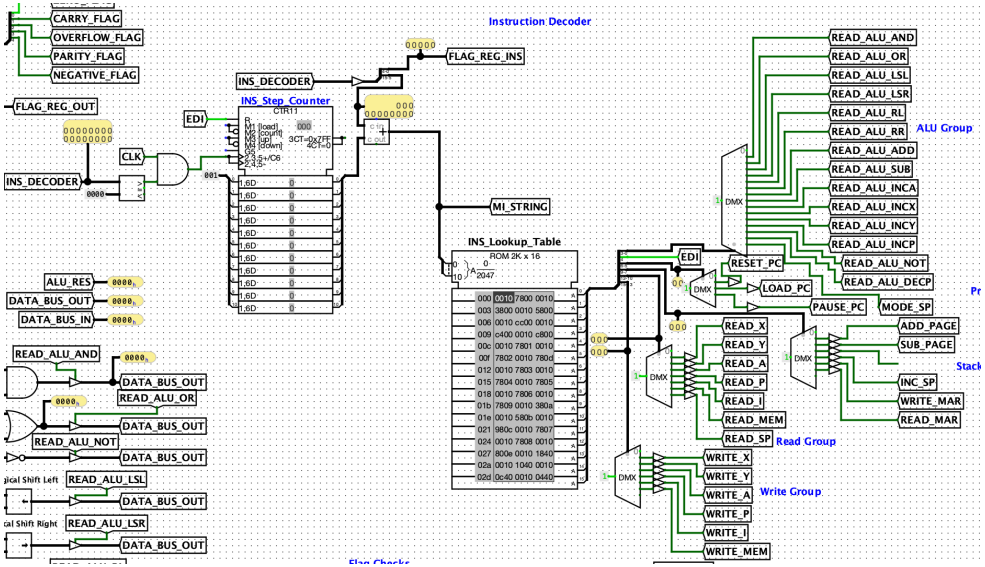






Control Unit

- Takes the instruction stored in the Instruction Register and breaks it into parts.
- Instruction = 11 most significant bits
- Flag Checks = 5 least significant bits
- Instruction part loaded into the Instruction Lookup Table



Six groups of microcode:

Write	Read	Indirect	PC	EDI	ALU
011	110	100	00	0	1111

Concatenate these together to get a microinstruction:

0111101000001111 = 0x7a0f

Writing to Accumulator, reading from memory, moving the Stack Pointer, and changing the Stack Pointer mode all in one clock tick!

Instructions

- Live in memory and are sent to the Instruction Register to be processed in the Control Unit.
- Made up of a series of microinstructions
- Instruction broken into three parts:
  - Hex Digit 1 = Addressing Mode
  - Hex Digit 2 & 3 = Lookup Table Index
  - Hex Digit 4 = Flags to Compare

LDX Immediate (0x0020)

STA PageAhead (0x5200)

ADC Absolute (0x453)

JMP Indirect (0x7660)

PSHY Implied (0x0700)

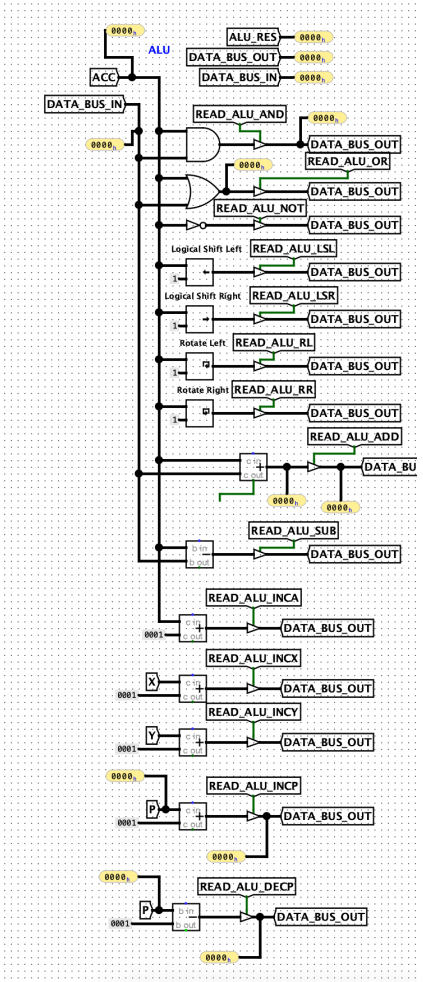
Addressing Modes

There are 8 addressing modes in the Guin-16:

1. **Implicit** - No data needed for this instruction.
2. **Immediate** - Next address consecutive address in memory holds the data.
3. **Page Ahead** - Data found at the address a page ahead of instruction.
4. **Page Behind** - Data found at the address a page behind the instruction.
5. **Absolute** - Data at next address points to the address of the data to use.
6. **AbsoluteX** - Add the contents of X to absolute address found.
7. **AbsoluteY** - Add the contents of Y to absolute address found.
8. **Indirect** - Perform two absolutes to get the data needed.

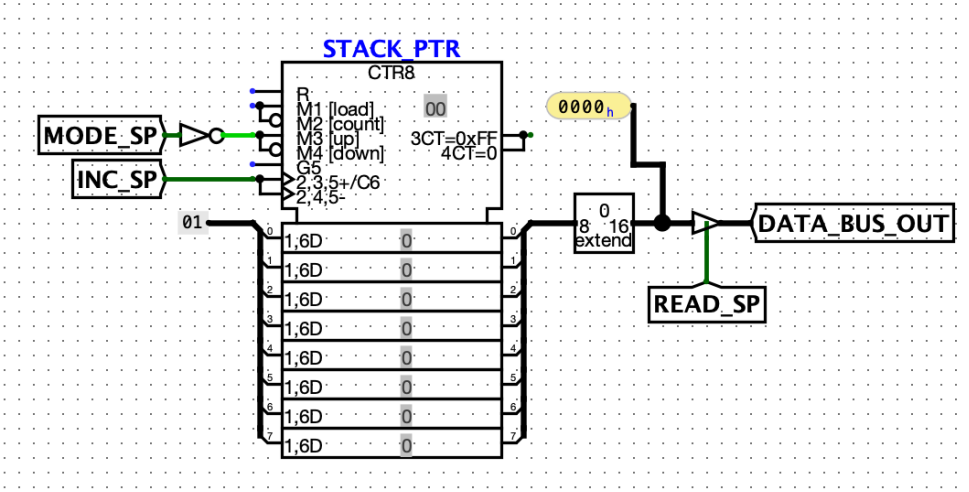
ALU

- Handles arithmetic and logical operations in the CPU
- Two-operand operations *always* use Accumulator
- Intermediate operations sent to Accumulator



Stack Pointer

- Points to the current top of the Stack
- Increments every push operation
- Decrements every pop operation



Documentation & Demonstration